

Guía de Laboratorio Avanzada de POO

EVALUACIÓN PRÁCTICA: AUDITORÍA DE COLECCIONES Y CONTROL DE EXCEPCIONES

Asignatura: Programación Orientada a Objetos | **Profesor:** Edilberto Lacayo | **Dinámica:** Cacería a Código Abierto (10 Bugs)

*¡Hola, muchachos! Bienvenidos a su sesión práctica de aseguramiento de calidad de software en POO. Ya dominamos las estructuras de datos nativas de Python y cómo operan dentro de nuestras arquitecturas de clases. Hoy nos enfrentaremos a un reto real: el siguiente script diseñado para controlar las calificaciones de un centro educativo contiene exactamente ****10 fallas de lógica y semántica ocultas****, sin ningún comentario que delate su ubicación. Estudien las instrucciones, ejecuten las pruebas y completen el reporte final siguiendo los requisitos de entrega.*

1. Código Fuente del Sistema (Limpio de Comentarios de Guía)

Copien textualmente este script funcional en su entorno de desarrollo local y ejecútenlo de forma secuencial para observar y analizar los puntos de detención del intérprete:

```

class Estudiante:
    def __init__(self, id_estudiante, nombre):
        self.id_estudiante = id_estudiante
        self.nombre = nombre
        self.record_notas = {}

    def agregar_nota(self, asignatura, nueva_nota):
        if asignatura in self.record_notas:
            self.record_notas[asignatura].add(nueva_nota)
        else:
            self.record_notas[asignatura] = [nueva_nota]

class ControlEscolar:
    def __init__(self, nombre_colegio, coordenadas_gps):
        self.auditoria = (nombre_colegio, coordenadas_gps)
        self.registro_estudiantes = set()
        self.historial_acciones = []

    def matricular_estudiante(self, estudiante):
        self.registro_estudiantes.append(estudiante)
        self.historial_acciones.append(f"Matriculado: {estudiante.nombre}")

    def actualizar_nombre_colegio(self, nuevo_nombre):
        self.auditoria[0] = nuevo_nombre

    def obtener_estudiante_por_posicion(self, posicion):
        return self.registro_estudiantes[posicion]

    def eliminar_primer_evento_historial(self):
        if len(self.historial_acciones) > 0:
            self.historial_acciones.remove(0)

    def extraer_calificaciones(self, estudiante, asignatura):
        return estudiante.record_notas[asignatura]

    def limpiar_historial_completo(self):
        self.historial_acciones = tuple()

# === SCRIPT DE EJECUCIÓN DE PRUEBAS CONTROLADAS ===
if __name__ == "__main__":
    colegio = ControlEscolar("Instituto Técnico Occidental", ("12.43 N", "86.87 W"))

    alumno1 = Estudiante("EST_01", "Anthony")
    alumno2 = Estudiante("EST_02", "Nayeli")

    colegio.matricular_estudiante(alumno1)
    colegio.matricular_estudiante(alumno2)

    alumno_clon = Estudiante("EST_01", "Anthony Cáceres")
    colegio.matricular_estudiante(alumno_clon)

    alumno1.agregar_nota("Programación", 85)
    alumno1.agregar_nota("Programación", 92)

    alumno_buscado = colegio.obtener_estudiante_por_posicion(0)
    print(f"Estudiante recuperado: {alumno_buscado.nombre}")

```

```
notas_bd = colegio.extraer_calificaciones(alumno2, "Bases de Datos")
```